

캡스톤디자인 중간발표

RustDB

Custom RDBMS Project

발표자 김규현, 빌궁
학과 소프트웨어학과

프로젝트 주제

주제

커스텀 RDBMS 시스템 개발

내용

AI 기반 커스텀 RDBMS 시스템

SQL 쿼리 작성 및 실행, 데이터 저장 및 복구, 다중 사용자 실시간 시스템, GUI 기반 테이블 생성 및 조작, 자연어 기반 AI 자동 쿼리 작성 기능 등을 지원하는 Rust로 개발된 풀스택 RDBMS 시스템

프로젝트 기능

No. 1

RDBMS 시스템

관계형 데이터베이스에서 SQL 쿼리문 작성을 통해 데이터를 생성·조회·수정·삭제 등을 할 수 있는 데이터베이스 엔진 기능

No. 2

네트워크 서버

TCP 기반 서버를 생성하고, 인가된 다중 사용자가 데이터베이스에 접근하여 동시에 데이터를 조작할 수 있도록 하는 실시간 시스템 기능

No. 3

GUI

GUI를 기반으로 SQL 쿼리문 없이 화면에서 테이블을 생성 및 조작하고, 관계를 설정할 수 있는 시각적 편집기 기능

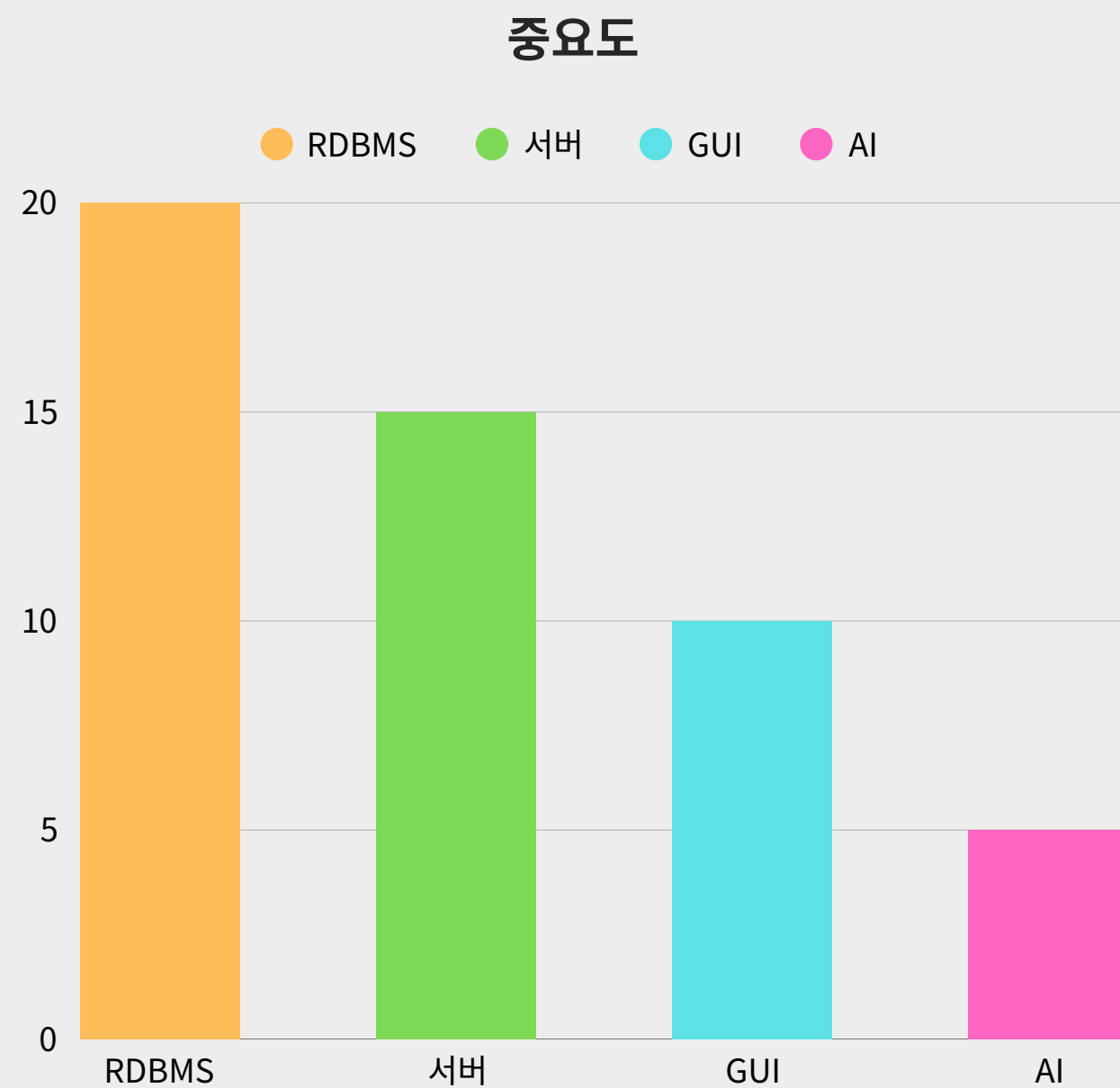
No. 4

AI

인공지능의 MCP 기술을 기반으로 비전문 사용자도 SQL 쿼리 작성 없이 자연어를 통해 데이터를 조작할 수 있도록 하는 기능

프로젝트 평가 계획

4가지 핵심 기능을 안전하고 완전하게 사용 가능한 형태를 완성 기준으로 삼고 평가한다.



● RDBMS

실제 SQL 쿼리문을 작성하고 실행했을 때 정확한 결과가 나오는가
데이터 변경 중 오류가 발생했을 때 데이터가 안전하게 복구되는가

● 서버

여러 사용자가 동시에 접속해서 같은 데이터베이스를 다룰 수 있는가
암호 등을 통해 인가받은 사용자만이 데이터베이스에 접근할 수 있는가

● GUI

SQL 쿼리 없이도 시각적으로 테이블을 만들고 데이터를 조작할 수 있는가
테이블 간의 관계(FK)를 시각적으로 직접 설정하고 확인할 수 있는가

● AI

자연어로 질의 쿼리를 작성했을 때 원하는 쿼리문으로 작성해주는가
AI로 생성한 자연어 기반 쿼리문이 올바른 로직을 수행하는가

진행 상황

진행중인 기능

1. 데이터베이스 엔진 고도화 및 최적화 작업
2. 네트워크 서버 인증 기능
3. GUI 기반 테이블 편집기 설계
4. UI/UX 개선 등

완료된 기능

1. 데이터베이스 엔진 구현 완료
2. 데이터 저장&복구 및 트랜잭션
3. UI/UX 클라이언트 70% 완성
4. 네트워크 서버 구현(TCP 포트 7878) 등

가능성 확인 완료

1. GUI Table Editor : React Flow 기반 시각적 편집기 구현 가능성 확인 (라이브러리 검토 완료)
2. AI 연동 : AI MCP API 연동 방식 및 SQL 변환 흐름 설계 가능성 확인 등

05

산출물

RustDB

MCP 기반 커스텀 RDBMS

프로젝트 문서 통합본 | v14 | 김규현, 발웅 | 2026년

번호	문서명	영문명
1	업무 분석표 (간트 차트)	Work Breakdown Structure (Sanrtt Chart)
2	서론 (개발 목표, 개발 내용)	Introduction
3	요구사항 정의	Requirements Definition
4	사용자 인터페이스 (UI) 명세	User Interface Specification
5	기능적 모델링: Use-Case Diagram	Functional Modeling: Use-Case Diagram
6	구조적 모델링: Class Diagram 및 클래스 정의	Structural Modeling: Class Diagram and Class Definitions
7	클래스 메소드 알고리즘 명세	Method Algorithm Specification
8	주요 Use-Case 시퀀스 다이어그램	Sequence Diagram for Major Use-Cases

1. 업무 분석표 (간트 차트)

Work Breakdown Structure (Sanrtt Chart)

대분류	작업 항목	WS1	WS2	WS3	WS4	WS5	WS6	WS7	WS8	WS9	WS10	WS11	WS12	WS13	WS14	WS15	WS16	WS17	WS18	WS19	WS20
환경설정	개발환경 구성 (Rust VS Code)																				
제어	Lexer / Tokenizer																				
해석	Parser / AST																				
Executor	Executor (AST 실행기)																				
DOL	CREATE TABLE / DROP TABLE / TRUNCATE																				
DOL	ALTER TABLE (ADD/DROP/RENAME/ADDINDEX)																				
DOL	CREATE / DROP INDEX (인덱스)																				
DOL	CREATE / DROP VIEW / DESCRIBE																				
DOL	DATETIME / TIMESTAMP 처리 (기능 명세)																				
DOL	TIME / YEAR / MONTH / SET 처리																				
DOL	다중 테이블에서 CREATE / DROP / USE DATABASE																				
DML	INSERT / UPDATE / DELETE																				
DML	SELECT / WHERE / ORDER BY / LIMIT / DISTINCT																				
DML	JOIN (INNER/LEFT/RIGHT) / WITH 조인																				
DML	GROUP BY / HAVING / 집계함수																				
DML	서브쿼리 (SUBQUERIES) 포함																				
DML	스칼라함수/NULL 함수 / EXPLAIN / ANALYZE																				
DML	순차 함수 (ROUND/ABS/CEIL/FLOOR/MOD)																				
DML	UPDATE / DELETE 다중 테이블																				
데이터베이스	PK / UNIQUE / NOT NULL / DEFAULT / CHECK																				
데이터베이스	AUTO_INCREMENT / ON DELETE / ON UPDATE																				
DOL	WAL (Write-Ahead Log) / Check Recovery																				
DOL	BEIN / COMMIT / ROLLBACK / SAVEPOINT																				

2. 서론

Introduction

2.1 개발 목표

RustDB는 Rust 언어 기반의 관계형 데이터베이스 관리 시스템(RDBMS)으로, 데이터베이스 엔진의 핵심 구성 요소를 기호부터 설계하고 구현하는 것을 핵심 목표로 삼는다. SQL 어휘 분석 및 구문 분석, 실행 계획, 스토리지 엔진에 이르기까지 전 계층을 하나의 시스템으로 통합하여, B-Tree 인덱스, WAL 기반 장애 복구, 트랜잭션 격리 수준, 행 단위 잠금, 데드락 감지 등 데이터베이스 이론의 핵심 개념들이 설계 목적하는 시스템으로 구현되도록 한다.

두 번째 목표는 Model Context Protocol(MCP) 기반의 AI 자연어 처리 기능을 통합하는 것이다. SQL 문법에 익숙하지 않은 사용자도 자연어만으로 데이터베이스를 조작할 수 있는 인터페이스를 제공함으로써, 데이터베이스 시스템의 접근성을 높인다. 아울러 타이널 REPL 클라이언트, TCP 기반 서버, 데스크톱 GUI 클라이언트를 모두 갖춘 올스택 데이터베이스 시스템으로서 완성도 높은 결과물을 목표로 한다.

항목	내용
프로젝트명	MCP 기반 커스텀 RDBMS (RustDB)
프로젝트 유형	풀스택 작품
개발 언어	Rust (엔진 / CLI / 서버), TypeScript / React (UI)
프레임워크	Tauri (데스크톱 앱), Monaco Editor, crossterm (CLI)
버전	v2.0.0
개발 기간	2026년 3월 ~ 2026년 6월

2.2 개발 내용

2.2.1 시스템 구성

RustDB는 Cargo Workspace 구조로 구성된 다섯 개의 크레이트로 이루어진다. 핵심 엔진 크레이트인 rustdb-core는 SQL 처리를 담당하는 파서 모듈, 처리 실행 및 잠금 관리를 담당하는 엔진 모듈, 데이터 저장 및 인덱싱을 담당하는 스토리지 모듈, 트랜잭션 및 WAL 로그를 관리하는 트랜잭션 모듈, 데이터를 스키마로 관리하는 커탈로그 모듈로 구성된다. AI 자연어 처리 기능은 별도 크레이트 rustdb-mcp로 분리하여 개발 예정이다.

인터페이스 계층에서는 세 가지 클라이언트를 제공한다. rustdb-cli는 터미널 기반 REPL로 SQL 입력과 결과 출력에 지원하여, rustdb-server는 포트 7878에서 동작하는 TCP 서버로 다중 클라이언트 동시 접속을 처리한다. rustdb-ui는 Tauri와 React를 결합한 데스크톱 GUI 애플리케이션으로, VSCode 스타일의 인터페이스와 Monaco 에디터를 통해 직관적인 SQL 작성 환경을 제공한다.

컴포넌트	역할	기술 스택
rustdb-core	DB 엔진 라이브러리 (파서 / 실행 / Rust, serde, serde_json, chrono)	

7. 클래스 메소드 알고리즘 명세

Method Algorithm Specification

7.1 Lexer:tokenize()

항목	내용
클래스	Lexer
메서드	tokenize() -> Vec<Token>
목적	SQL 문자열 전체를 순차적으로 읽어 표준 토큰으로 변환

- pos 가 input 끝을 도달할 때까지 반복
- 공백 -> 건너뛰기 / 일파열 -> read_ident() (키워드 또는 ident) / 숫자 -> read_number() / ' -> read_string()
- 연산자/구분자 -> 대응 토큰 / -- 또는 # -> 줄 끝 주석 건너뛰기 / /* ... */ -> 블록 주석 건너뛰기

7.2 Parser:parse_condition()

항목	내용
클래스	Parser
메서드	parse_condition() -> Result<Condition, String>
목적	WHERE 절 조건식 파싱 (IS NULL, BETWEEN, LIKE, IN, EXISTS, 서브쿼리 포함)

- 합동명(=는 table.column) 파싱
- peek()가 is -> IS NULL (ISNULL) 또는 IS NOT NULL (ISNOTNULL)
- BETWEEN -> start AND end -> between(start, end) / LIKE -> 패턴 문자 -> Operator::Like
- IN / NOT IN -> 서브쿼리 또는 테이블 목록 / EXISTS / NOT EXISTS -> 서브쿼리 파싱
- 나머지 -> 일반 비교 연산자 (<, >, <=, >=, <>)
- parse_condition_chain()으로 and Option<Box<Condition>> / or Option<Box<Condition>> 재귀 처리

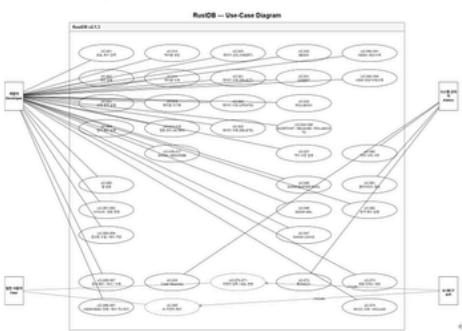
7.3 Executor:exec_insert()

항목	내용
클래스	Executor
메서드	exec_insert(table, columns, values) -> Result<String, String>
목적	INSERT 실행 - 읽기 row, 제약조건 검사, WAL 로깅

5. 기능적 모델링: Use-Case Diagram

Functional Modeling: Use-Case Diagram

5.1 Use-Case Diagram



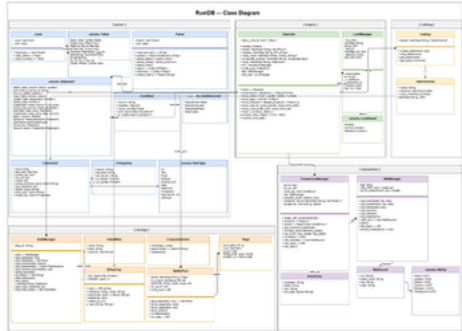
5.2 액터 정의

액터	유형	설명	주요 상호작용
개발자 (Developer)	주요 액터	SQL을 직접 입력하는 기술 사용자	CLI, GUI 에디터, TCP 클라이언트
일반 사용자 (User)	주요 액터	자연어로 데이터를 조회하는 비기술 사용자	AI 자연어 처리 (예제)
시스템 관리자 (Admin)	보조 액터	서버 및 데이터 관리	TCP 서버 시작/종료
AI MCP API (Externally)	외부 시스템	자연어를 SQL로 변환하는 외부 AI	MCP API 호출 (예제)

6. 구조적 모델링: Class Diagram 및 클래스 정의

Structural Modeling: Class Diagram and Class Definitions

6.1 Class Diagram



6.2 클래스 정의

6.2.1 파서 객체 (src/parser/)

Lexer (Struct)

구분	항목	설명
위치	src/parser/lexer.rs	
역할	SQL 문자열을 토큰 스트림으로 변환	
필드	input: Vec<char>	입력 문자 배열
필드	pos: usize	현재 파싱 위치
메서드	tokenize() -> Vec<Token>	전체 토큰 리스트 반환
메서드	read_ident / read_number /	식별자/숫자/문자를 표준 토큰 읽기

3. 요구사항 정의

Requirements Definition

3.1 기능적 요구사항

3.1.1 SQL 파싱 및 실행

요구사항 ID	요구사항명	설명	상태
FR-001	Lexer 구현	SQL 문자열을 키워드/식별자/연산자/리터럴 토큰으로 분리	완료
FR-002	Parser 구현	토큰 스트림을 AST로 변환 (재귀 하강 파싱)	완료
FR-003	Executor 구현	AST를 해석하여 실제 데이터 조작 실행	완료
FR-004	월리 처리 / 주석 처리	-- 코 주석된 텍스트 처리, /* ... */ 주석 지움 (문자열 주석 내 세미콜론 안전 처리)	완료

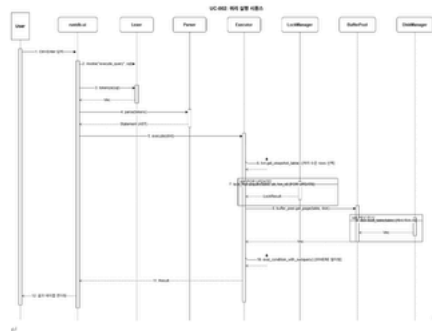
3.1.2 DDL 요구사항

요구사항 ID	요구사항명	설명	상태
FR-010	CREATE TABLE	합동 명치 및 제약조건 포함 테이블 생성 (IF NOT EXISTS, 복합 PK, CHECK)	완료
FR-010b	다중 데이터베이스	CREATE / DROP DATABASE (IF EXISTS) USE (database), 데이터베이스 간 완전한 데이터 격리	완료
FR-011	DROP TABLE	테이블 및 관련 데이터 삭제 (IF EXISTS 포함)	완료
FR-012	TRUNCATE TABLE	테이블 구조 유지, 데이터 전체 삭제, AUTO_INCREMENT 리셋	완료
FR-013	ALTER TABLE	ADD / DROP / RENAME / MODIFY COLUMN / RENAME TO (테이블 이름 변경)	완료
FR-014	CREATE / DROP INDEX	B-Tree 기반 인덱스 / 복합 합동 인덱스 생성 및 삭제 (IF EXISTS 포함)	완료
FR-015	CREATE / DROP VIEW	SELECT 기반 가상 테이블 생성 및 삭제 (IF EXISTS 포함)	완료
FR-016	DESCRIBE	테이블 스키마 구조 조회	완료

8. 주요 Use-Case 시퀀스 다이어그램

Sequence Diagram for Major Use-Case

8.1 UC-002: 쿼리 실행 시퀀스



단계	참여자	메시지	설명
1	User	Ctrl+Enter 입력	쿼리 실행 트리거
2	rustdb-ui	invoke("execute_query", sql)	Tauri invoke 호출
3	Lexer	tokenize(sql)	SQL -> 토큰 리스트 (Vec<Token>)
4	Parser	parse(tokens)	토큰 -> AST (Statement)
5	Executor	execute(stmt)	AST 분기 처리
6	Executor	txn.get_snapshot_table()	각기 수준에 따른 rows 선택
7	Executor -> LockManager	lock_mgr.acquire_table_pk, txn_id	FOR UPDATE 시 락 잠금
8	Executor -> BufferPool	buffer_pool.get_page(table, disk)	캐시에서 rows 조회

4. 사용자 인터페이스 (UI) 명세

User Interface Specification

4.1 전체 UI 구조

rustdb-ui는 Tauri + React 기반 데스크톱 앱으로, VSCode 스타일의 레이아웃을 채택하며, 에디터 뷰 / GUI 데이터를 브라우저 뷰 / TCP 서버 관리 뷰 세 가지 액티브 뷰를 지원한다.

영역	구성요소	설명
액티비티 바	아이콘 버튼 (피크, 세팅)	에디터 / GUI 브라우저 / 서버 관리 탭 전환
사이드바	데이터베이스/쿼리 트리	드래그로 너비 조절, 색인 추가/제거, 경우 몇 지
에디터 영역	Monaco Editor	SQL 입력, 문법 강조, Ctrl+Enter 실행
결과 영역	결과 데이터	월리 쿼리 결과 순서대로 표시
구분선	드래그 핸들	에디터/결과 영역 높이 조절
GUI 뷰	데이터 그리드 + 필터	데이터를 데이터 브라우저, 탭 필터링
서버 관리 뷰	TCP 서버 제어	서버 시작/중지, 포트 설정, 로그 실시간 표시
AI Assistant 뷰	AI 질문 인터페이스	자연어 쿼리 입력, AI API 연동 (예제), 데이터용 완성

4.2 주요 화면 명세

4.2.1 메인 에디터 화면

컴포넌트	기능	상태
Monaco Editor	SQL 문법 강조, 주석 처리, Ctrl+Enter 실행	완료
선택 실행	드래그 선택 영역만 실행	완료
월리 쿼리	-- 코 주석된 텍스트 쿼리, 문법 강조, 순서대로 표시	완료
쿼리 자동 저장	localStorage에 자동 저장, 재실행 시 복제	완료
다중 쿼리 업	업 추가 / 전환 / 닫기, 실행 쿼리 자동 저장	완료

4.2.2 사이드바

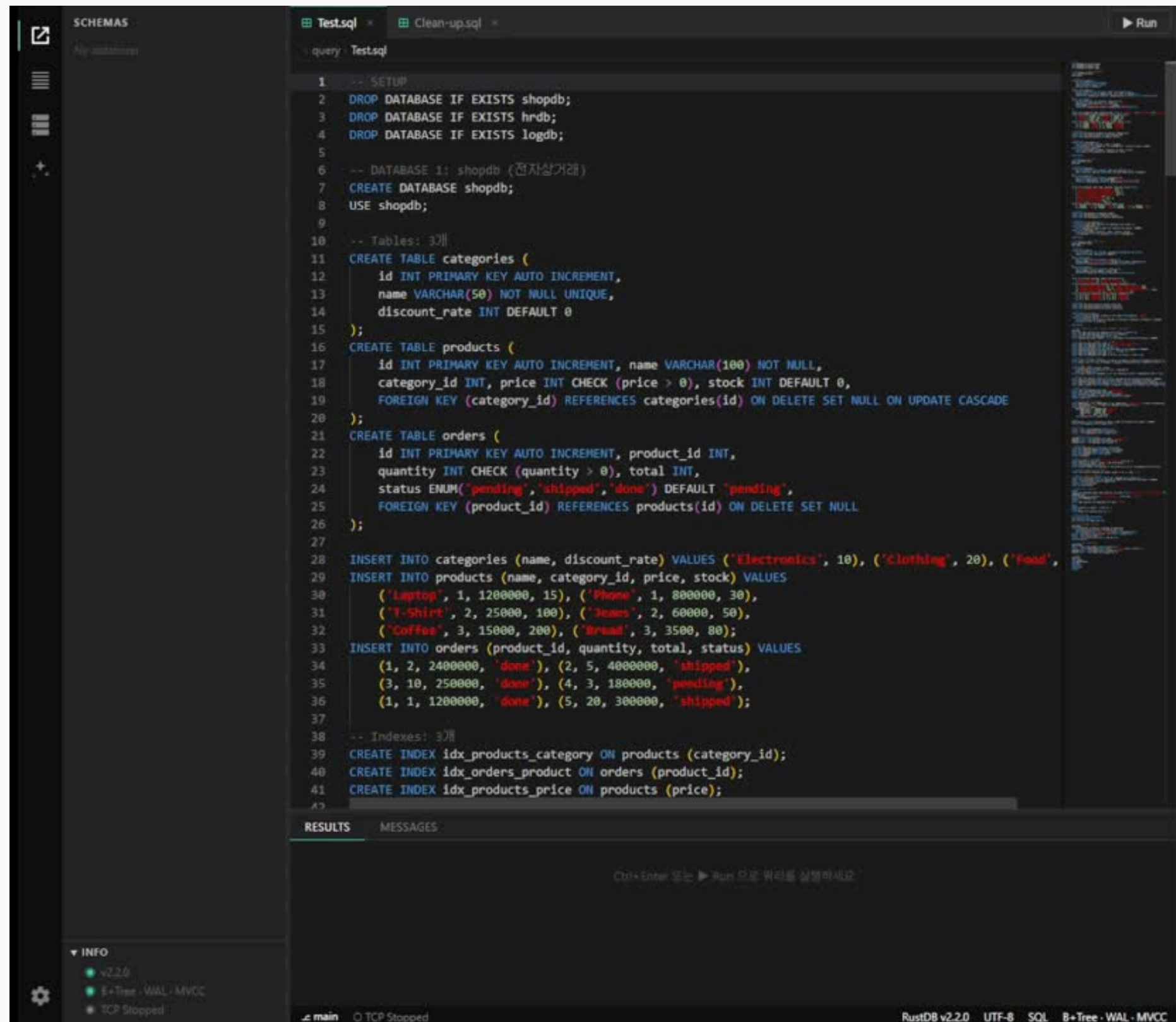
영역	기능	상태
다중 데이터베이스	Database > Tables/Views/Indexes 계층 구조, 여러 DB 동시 열리기	완료

현재 프로젝트는 WBS, 개발 목표 및 내용, 요구사항 정의, UI 명세, 기능적 모델링, 구조적 모델링, 클래스 메소드 알고리즘 명세, 주요 Use-Case에 대한 시퀀스 다이어그램 등을 작성한 프로젝트 통합한 문서로 관리하고 있다.

주로 문서에서 예정 섹션을 추가하여 작성하고 이에 따른 구현을 진행하고 있다.



중간 결과 동영상 시연



메인 페이지

DATABASE, TABLE, VIEW, INDEX

GUI 테이블 에디터

Visual Studio Code의 ERD Editor
와 비슷한 로직으로 시각적으로 테이블
을 수정할 수 있는 곳

TCP 네트워크 서버

포트 7878의 네트워크 서버를 생성하
고 다중 사용자를 관리할 수 있는 곳

AI 관련 기능 페이지

AI MCP 및 관련 기술을 응집해놓은 곳

향후 일정 계획

학기말까지 계획

1. 데이터베이스 엔진의 최적화 및 고도화 작업까지 전부 완료
2. 다중 사용자 인증 기능 포함 완전한 실시간 시스템 완성
3. GUI 기반 테이블 생성 및 조작 기능 설계
4. UI/UX의 기능 추가 및 개선

방학중 ~ 최종 캡스톤디자인2 계획

1. 데이터베이스 엔진의 추가 개선점 탐색 및 수정
2. GUI 기반 시각적 편집기 기능 완성
3. AI MCP 기반 쿼리 작성 및 실행 기능 완성
4. 프로젝트의 UI/UX 최종 완성

Q & A